

Subject: SAST Summer Training 2026, Agent Harness

Date: from 2026/10/09 to 2026/10/10

Contents

Preliminaries and Run Trajectory	1
---	----------

Preliminaries and Run Trajectory

最小工具调用链路

可以用下面这条链路理解一次最小的“模型调用工具”的交互：从结构化请求出发，经过模板序列化交给模型，再把模型输出解析回结构化的工具调用。

JSON 请求

- > message / tool 对象
- > Chat Template 序列化为 prompt 文本或 token
- > 模型生成 response 文本
- > JSON.parse + Schema 校验
- > 统一的文本事件或 tool_use

例如，Harness 先准备结构化请求：

```
{
  "messages": [
    { "role": "system", "content": "You are a coding assistant." },
    { "role": "user", "content": "Read README.md" }
  ],
  "tools": [
    {
      "name": "read_file",
      "description": "Read a UTF-8 text file",
      "input_schema": { "type": "object", "properties":
        { "path": { "type": "string" } } }
    }
  ]
}
```

Chat Template 序列化

对于本地模型，Tokenizer 的 Chat Template 会把这些消息序列化成模型训练时使用的格式，示意如下（实际标记由模型决定）：

```
<|system|>
You are a coding assistant.
Available tools:
- read_file(path: string): Read a UTF-8 text file.
When you need a tool, output exactly:
<tool_call>{"name":"tool_name","arguments":{...}}</tool_call>
<|end|>
<|user|>Read README.md<|end|>
<|assistant|>
```

模型输出与解析

模型实际生成的是一段 token 流（从程序角度看是文本），例如：

```
<tool_call>
{"name":"read_file","arguments":{"path":"README.md"}}
</tool_call>
```

Harness 先识别并去掉外层标签，再解析标签内部的 JSON，最后做 Schema 校验：

```
const jsonText = responseText
  .replace('<tool_call>', '')
  .replace('</tool_call>', '')
  .trim()
const call = JSON.parse(jsonText)
validateToolInput(call.name, call.arguments)

const toolUse = {
  type: 'tool_use',
  id: 'toolu_01',
  name: call.name,
  input: call.arguments,
}
```

解析失败或 Schema 不匹配时，Harness 应返回错误、请求模型重试或终止本轮，而不是把未经校验的字符串交给 Shell 或文件系统。云端 API 往往在服务端完成 Chat Template 和工具调用封装；本地推理框架则可能在客户端完成，但对 Harness 来说，仍然是“结构化请求 → token/事件流 → 结构化工具调用”的同一条边界。

三种 API 格式

在发送请求时我们一般有三种 API 格式：Responses API、Chat Completions API 和 Anthropic Messages API，它们互不兼容。

1. Responses API.

```
curl --request POST \
  --url "${BASE_URL}/responses" \
  --header "authorization: Bearer ${API_KEY}" \
  --header "content-type: application/json" \
  --data "{
    \"model\": \"${MODEL}\",
    \"input\": \"Hello World\",
    \"stream\": false
  }"
```

2. Chat Completions API.

```
curl --request POST \
  --url "${BASE_URL}/chat/completions" \
  --header "authorization: Bearer ${API_KEY}" \
  --header "content-type: application/json" \
  --data "{
    \"model\": \"${MODEL}\",
    \"messages\": [
      {
        \"role\": \"user\",
        \"content\": \"Hello World\"
      }
    ]
  }"
```

3. Anthropic Messages API.

```
curl --request POST \
  --url "${ANTHROPIC_BASE_URL}/messages" \
  --header "authorization: Bearer ${API_KEY}" \
  --header "x-api-key: ${API_KEY}" \
  --header "anthropic-version: 2023-06-01" \
  --header "content-type: application/json" \
  --data '{
    "model": "'${MODEL}'",
    "max_tokens": 1024,
    "messages": [
      {
        "role": "user",
        "content": "Hello World"
      }
    ]
  }'
```

运行轨迹观察

在阅读源码前，可以先从运行轨迹观察一次任务如何经历模型请求、工具调用、权限判断和结果回传。这能帮助我们后面的静态代码结构与真实执行过程对应起来。

- 实验入口。Harness 运行轨迹；
- 本地运行。Claude Code 轨迹抓取仓库。